

+ 26

The State of Boost

Boost Is Where You Become the
Engineer You Want to Be

HAROLD J. BOTT JR.



20
26



The next hour, in three acts



The Craft

How Boost is made, and why it earns your trust.



The Work

What that craft has actually built.



The Road Ahead

Where Boost goes next, and how you come build it.



#include <coder_community>

STARRING DAVE ABRAHAMS • ANDREI ALEXANDRESCU • ARNAUD BECHER • HARRY BOTT • MARSHALL CLOW • GREG COLVIN • ALAN DE FREITAS • GABRIEL DOS REIS • VINNIE FALCO • GLEN FERNANDES • BRADEN GANETSKY • ION GAZTAÑAGA • NAT GOODSPEED • DOUG GREGOR
ALEKSEY GURTOVOY • HOWARD HINNANT • DAISY HOLLMAN • JAAKKO JÄRVI • NICOLAI JOSUTTIS • JON KALB • ANDREW KOENIG • BRYCE LELBACH • INBAL LEVI • ANDREW LUMSDAINE • JENS MAURER • JOAQUÍN M LÓPEZ MUÑOZ • SEAN PARENT • ROBERT RAMEY
NINA RANNS • RENÉ F RIVERA MORELL • JEREMY SIEK • JOHN SPICER • ALEX STEPANOV • BJARNE STROUSTRUP • LOUIS TATTA • JASON TURNER • MICHAEL WONG EDITED BY ALAN CHALICH • ELLY MICHAELS • RENEE SCHWARTZ • ADDITIONAL EDITING BY DOMINIC DIAZ-SMITH • KURT ENGFEHR
EXECUTIVE IN CHARGE OF PRODUCTION GEOFF BELL • CINEMATOGRAPHY BY COLLIER LANDRY ANIMATION & VFX LUC GORCZYNSKI & MATT KADISH • ORIGINAL MUSIC ZACHARY WALTER • DIRECTED AND PRODUCED BY COLLIER LANDRY & RAY NOWOSIELSKI • EXECUTIVE PRODUCERS VINNIE FALCO & LOUIS TATTA

ACT I

The Craft

How Boost is made, and why it earns your trust.

The Craft · The Work · The Road Ahead

ORIGINS

How a craft was really taught

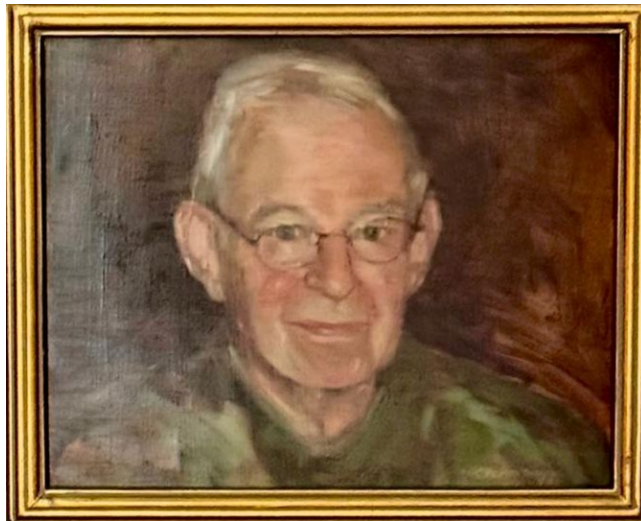
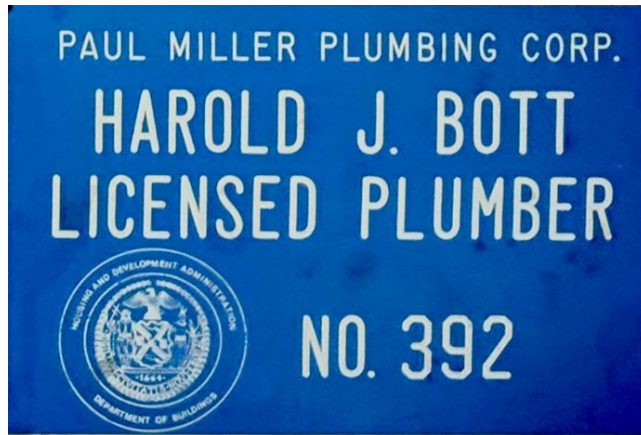
“Why did you do it that way?”

The first time a real engineer asked you that — and you had no answer. Most of us can name that person.



That question is how a craft is really taught.

ORIGINS · CONTINUED



Up From The Depths
The Daily News

Boost is the guild, rebuilt for our time

Documentation

tells you WHAT a library does.

A review

tells you WHY it is shaped that way: the three designs tried and abandoned, the compiler bug that forced the workaround, the decision the author still doubts.

You cannot get that from a book, a search box, or a model that has read everything and built nothing.

You use Boost every day. You just don't know it.

Now in the C++ standard.

`shared_ptr`

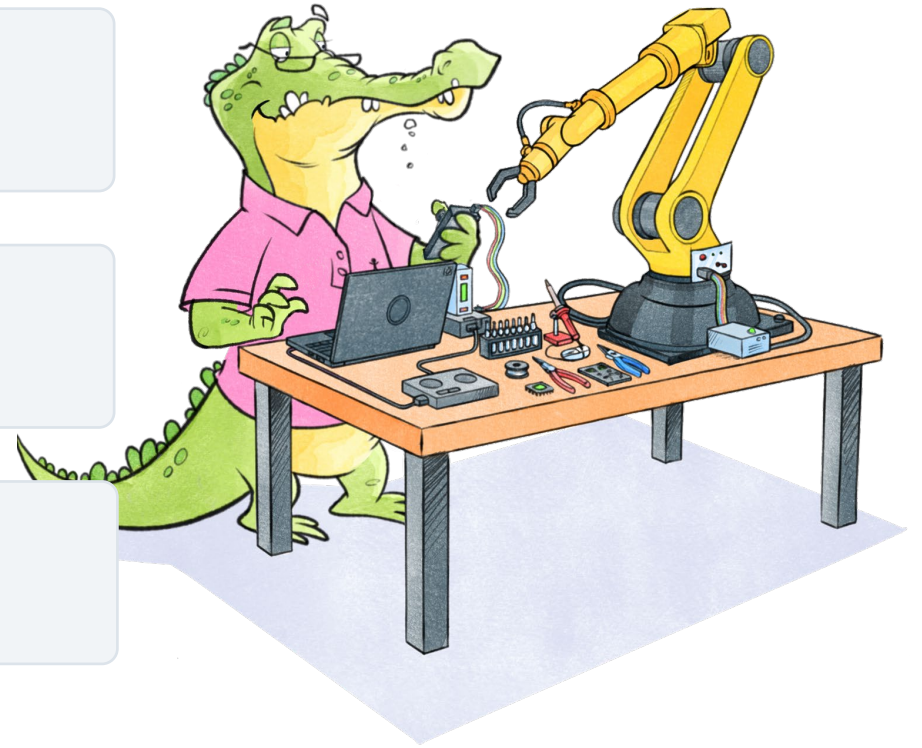
`optional`

`variant`

`filesystem`

`regex`

`type traits`



Every library that made it left its name at the door.

Name the builders



Peter Dimov

smart pointers, Mp11, decades of
core work



Howard Hinnant

<chrono>, move semantics, date



Marshall Clow

algorithms, string_view, long-time
maintainer



Beman Dawes

founder of Boost; Filesystem (in
memoriam)

Who replaces them?

3

CREDIBILITY ANCHOR

Not a random repo on GitHub

1

Formal peer review

A library earns entry by surviving public, expert review, not a merge button.

2

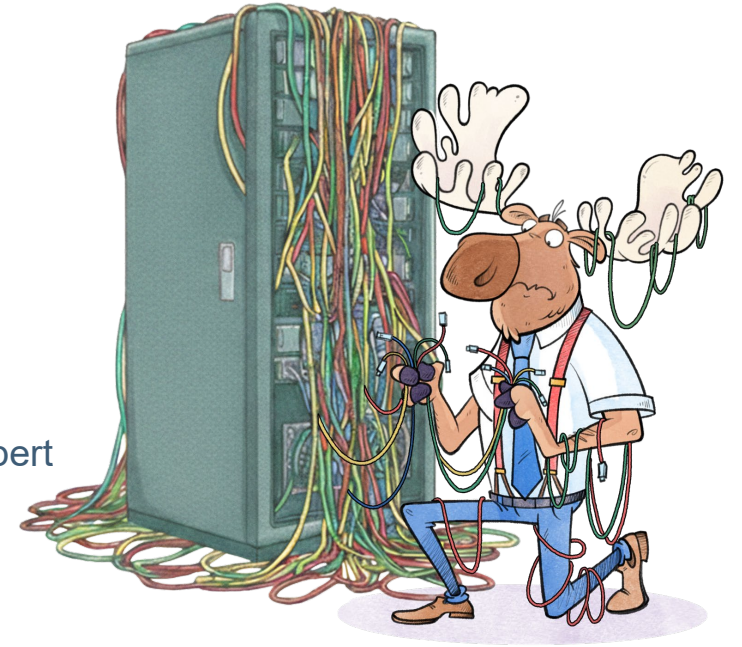
Production trust

Trading desks, automotive, aerospace ship Boost because it is vetted, not weekend code.

3

A staging ground

Boost is where the Standard Library's next features are tried in the real world first.



You already trust code built on Boost

*Close to **19,000 public repositories** pull in Boost, and **Asio** alone appears in **more than 3,200 of them**.*

BIG TECH & AI

Apple · Microsoft · Google ·
Amazon · Meta · IBM · Adobe ·
OpenAI

SEMICONDUCTORS

NVIDIA · AMD · Intel · Qualcomm
· Broadcom · Arm · Samsung ·
Analog Devices · Renesas ·
Microchip

FINANCE & PAYMENTS

JPMorgan Chase · Morgan
Stanley · Visa · Mastercard ·
Bloomberg

NETWORKING & INFRA

Cisco · Arista Networks · Juniper ·
Cloudflare · Canonical · Huawei

DATABASES & DATA ENGINES

MongoDB · Snowflake · Elastic ·
ClickHouse · ScyllaDB · ArangoDB
· DragonflyDB · PingCAP ·
Anyscale

CONSUMER INTERNET & MEDIA

Netflix · Spotify · Disney · Sony ·
Unity · Uber · Airbnb · eBay ·
Pinterest · Snap · Zoom · X

AEROSPACE & ENGINEERING

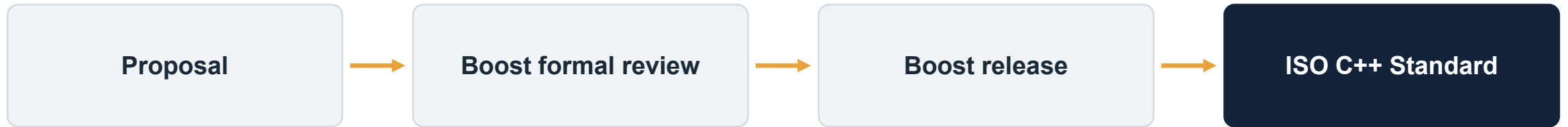
Boeing · Tesla · Siemens ·
Autodesk · MathWorks · Mapbox

GLOBAL INTERNET PLATFORMS

Tencent · Alibaba · ByteDance ·
Baidu · Meituan · DiDi · Yandex

Trademarks and names belong to their respective owners; shown for identification, not endorsement.

From Boost to the Standard, and back

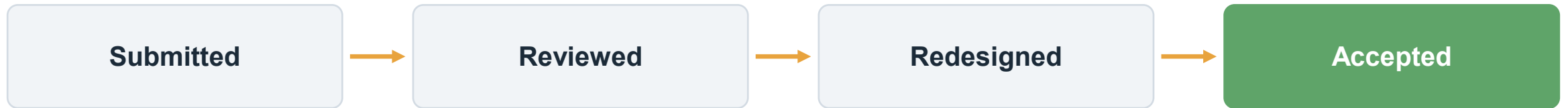


Graduated from Boost into the ISO Standard:

<code>smart pointers</code>	<code>filesystem</code>	<code>optional</code>	<code>variant</code>	<code>any</code>	<code>regex</code>	<code>type traits</code>	<code>chrono</code>
-----------------------------	-------------------------	-----------------------	----------------------	------------------	--------------------	--------------------------	---------------------

Boost stays relevant after graduation: backports, extended functionality, faster iteration.

Peer review is a forge, not a filter



Case study: Boost.Decimal

Entered **indeterminate** → ~9 months of rework → **Accepted**.

Authors: Matt Borland · Chris Kormanyos

Review manager: John Maddock (independent)

“I may well be being over-harsh in order to avoid bias.”

— John Maddock, the independent review manager, on reviewing colleagues' work

Boost.Decimal was written by people connected to the Alliance, and reviewed by an independent manager who bent over backwards against them. Disclosure turns 'capture' into a credential.

ACT II

The Work

What that craft has actually built.

The Craft · **The Work** · The Road Ahead

Everyone knew the map was slow

Two-step plan, early 2022:

1. **Win inside the rules:** Make Boost's own conforming map as fast as the standard allows.
2. **Leave the rules: Open addressing.** Compete with the tables people actually use.

Three names:

Peter Dimov

plan writer; internal reviewer

Joaquín López Muñoz

containers engineer; three Boost libraries author

Christian Mazakas

coauthor on boost.unordered



Step one shipped in Boost 1.80. The next slide is step two.

A small team out-measured the field

At $n \approx 1\text{M}$

5.67 ns

Boost unordered_flat_map

7.50 ns

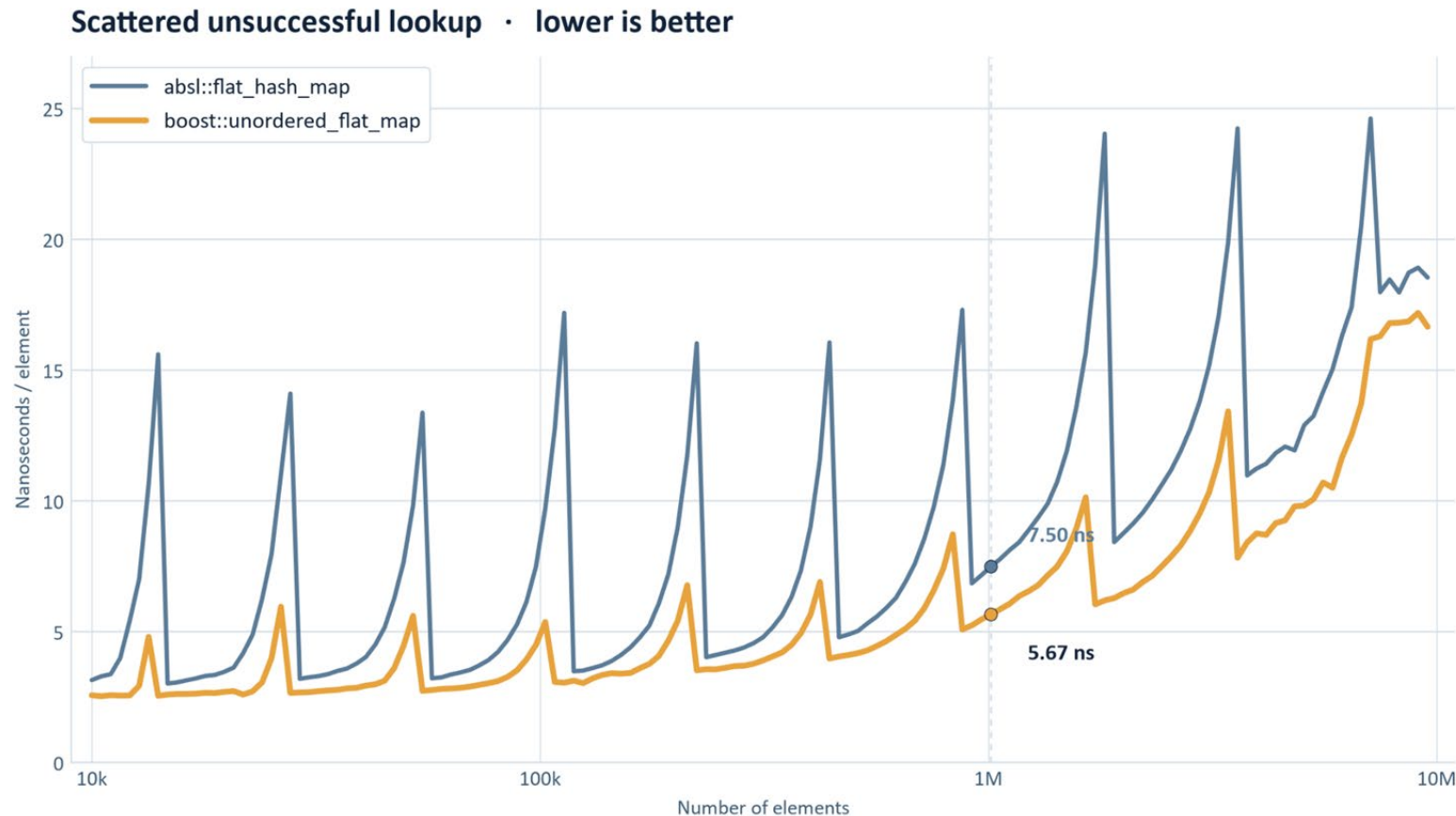
Abseil flat_hash_map

Average, 10k-10M

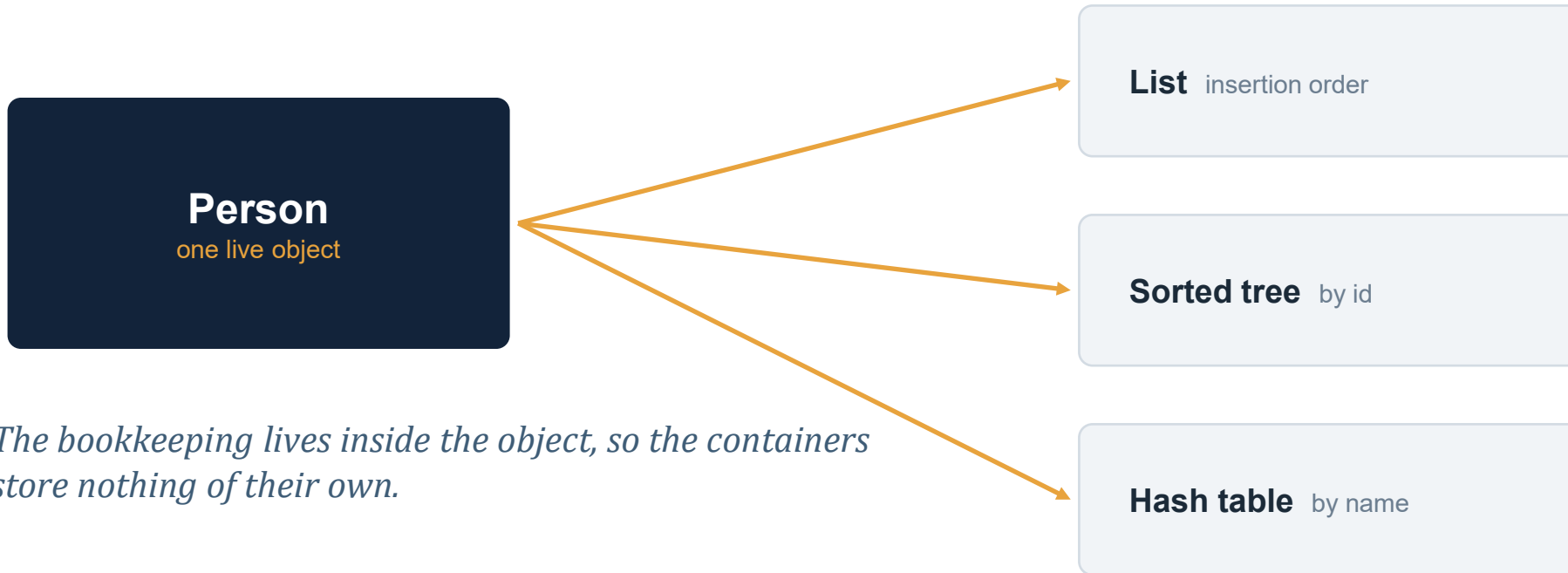
1.5x faster

Boost wins at all 142 measured sizes.

GCC 12, x64 · unordered_flat_map · unsuccessful lookup (a miss). Boost leads at every size here; on successful lookup and insertion, Abseil wins at some sizes.



One object. Three containers. Zero extra allocations.

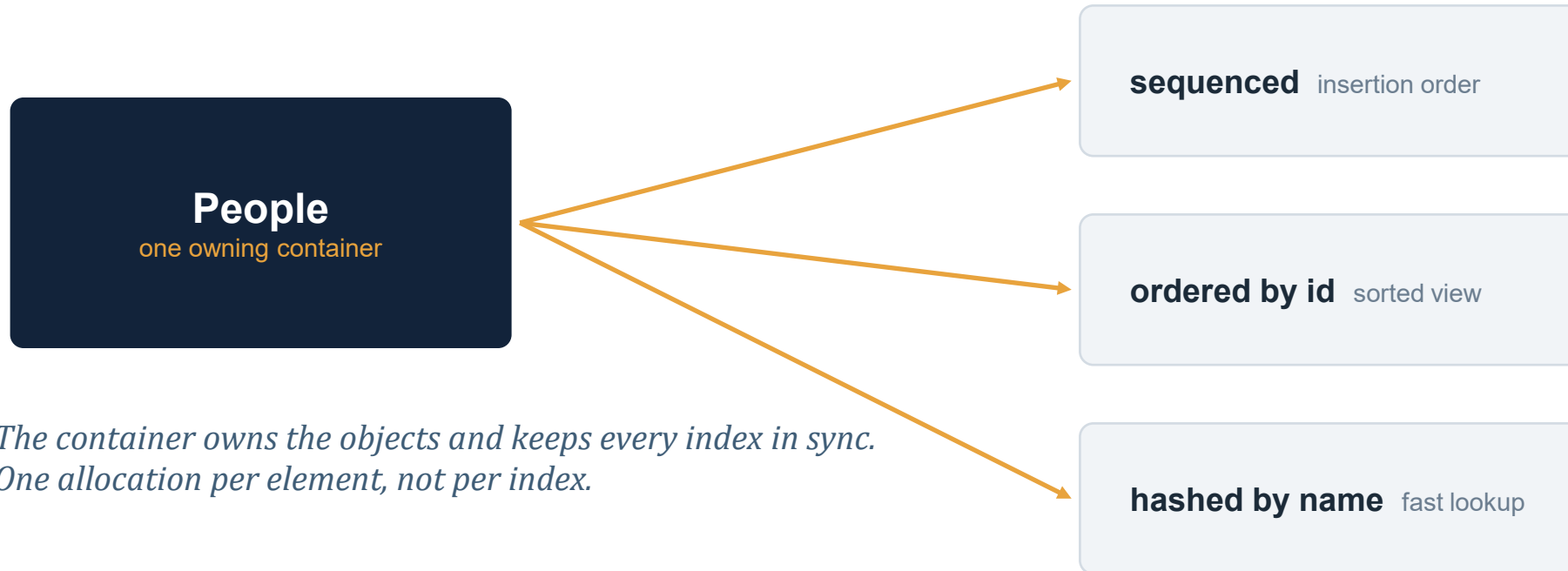


```
struct Person
: list_base_hook<>           // a list
, set_base_hook<>           // a sorted tree
, unordered_set_base_hook<> // a hash table
{ int id; std::string name; };
```

5

WHAT THEY BUILT · THE SAME TRICK, MADE EASY

Boost.MultiIndex: one container, every index



*The container owns the objects and keeps every index in sync.
One allocation per element, not per index.*

```
using People = multi_index_container<Person, indexed_by<
    sequenced<>,                                // insertion order
    ordered_unique< member<Person,int,&Person::id> >, // by id
    hashed_non_unique< member<Person,string,&Person::name> > // by name
>>;
```

Boost.Intrusive vs Boost.MultiIndex

Both solve the same problem:

Boost.Intrusive

Boost.MultiIndex

Who owns the objects

You do: containers are non-owning

The container does

Extra allocations

Zero: hooks live in your object

One node per element (all indices share it)

Lifetime management

You do it, carefully

Automatic

The feel

Low-level plumbing, max control

Ready-to-use, one declaration

Seen in the wild

Folly, Ceph

Broad application code

Since · author

2008 · Ion Gaztañaga

2004 · Joaquín López Muñoz

Same job, two tools: reach for MultiIndex when you want it done for you; reach for Intrusive when every allocation counts. Both peer-reviewed, both battle-tested. Boost hands you the choice.

Computers cannot add money

```
double:          0.10 + 0.20 == 0.30000000000000000004
decimal64:       0.10 + 0.20 == 0.30
```

Boost.Decimal: IEEE 754 decimal floating point that feels like a native type. Exact, and portable.

Six more, one line each

Boost.JSON

fast, safe JSON parsing

Boost.URL

correct URL parsing and building

Boost.Scope

scope guards for clean teardown

Boost.Cobalt

a coroutine framework

Boost.Hash2 + Describe

extensible hashing, easier with C++26 reflection

Boost.Bloom

fast, configurable Bloom filters for probabilistic set membership testing

ACT III

The Road Ahead

Where Boost goes next, and how you come build it.

The Craft · The Work · **The Road Ahead**

Networking: compile once, swap the transport

and then build whole protocols on top of it.

THE FOUNDATION · CAPY

your code → **any_stream** (type-erased) → **TCP · TLS · test mock**

Capy is the type-erasure of C++20 coroutines. One program runs over any transport, unchanged, at zero per-operation cost. External users already ship on it.

THE PAYOFF · HIGH-LEVEL PROTOCOLS

The Alliance's new HTTP library (not yet proposed for Boost) is built on Capy alone: server, parser, serialization, algorithms.

serve_static("./site") serves a whole website over HTTP. The idiomatic-C++ answer to Node's *serve-static*. No socket code.

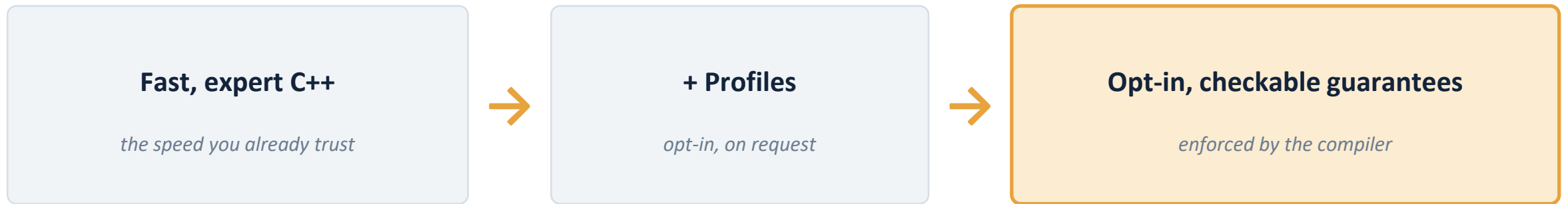


Beast (HTTP & WebSocket, mature) · **Capy** (the coroutine foundation) · **Corosio** (coroutine-native networking)

“Just use coroutines, bro.” — Peter Dimov to Vinnie Falco

Sept 2026: Capy and Corosio conditionally accepted into Boost after formal peer review, thanks to **Jeff Garland**, who ran the Asio review twenty years ago.

Where Boost goes next: guarantees you can opt into



Whole classes of errors ruled out by construction. The same craft, now with guardrails you can turn on.

Boost brings the guarantees as well as the speed.

THE ROAD AHEAD

<https://godbolt.org/z/ejM8Y8ra1>

```
COMPILER EXPLORER
Add... More Templates
C++ source #1
A Save/Load + Add new... Vim CppInsights Quick-bench C++
1 [[profiles::enforce(std::init)]]; // opt in
2 #include <cstdlib>
3 #include <memory> // construct_at is [[now_init]], destroy_at [[now_uninit]]
4
5 struct Slot { int value = 0; };
6
7 void demo() {
8     Slot *p [[ref_to_uninit]] = (Slot *)malloc(sizeof(Slot)); // raw memory, and p says so
9     p->value = 1; // error: uninit_write - no piecemeal init
10    std::construct_at(p); // [[now_init]]: initialize
11    p->value = 1; // OK: an ordinary
12    std::destroy_at(p); // [[now_uninit]]: destroy
13    int v = p->value; // error: uninit_read - the storage is dead
14    std::destroy_at(p); // error: double_destroy
15    free(p);
16 }
17
```

Output of x86-64 clang (std::init profile - P4222) (Compiler #1)

A Wrap lines Select all

```
<source>:9:12: error: writing a member of uninitialized storage reached through a
'[[ref_to_uninit]]' pointer or reference does not initialize it under profile 'std::init';
initialize the whole object ('construct_at()' for a class object)
    9 |   p->value = 1; // error: uninit_write - no piecemeal init
      |           ^

<source>:13:14: error: read through a '[[ref_to_uninit]]' pointer or reference accesses
uninitialized memory under profile 'std::init'
    13 |   int v = p->value; // error: uninit_read - the storage is dead
      |           ^

<source>:14:19: error: storage already destroyed by a '[[now_uninit]]' function is destroyed
again under profile 'std::init'
    14 |   std::destroy_at(p); // error: double_destroy
      |           ^

3 errors generated.
Compiler returned: 1
```

We are building the whole toolchain

The language only moves as fast as the compilers underneath it.

THE PROBLEM

C++ moves only as fast as the compilers under it. Very few engineers in the world can work inside one, and that is a bottleneck for the whole language.

WHAT WE ARE DOING

The C++ Alliance Compiler Initiative: a mentored pipeline into Clang. Real engineers, real patches landing upstream, a repeatable way to grow compiler people. GCC is next.

Library authors, standards people, and compiler people, pulling in the same direction, in the open.

The pipeline, with a face



Matheus Izvekov

Staff Engineer, C++ Alliance · long-time Clang contributor · on the C++ committee

He maintains one of the hardest parts of Clang: **the C++ template engine and the AST**, the internal machinery every one of your builds runs through.

Why you care, even if you never touch a compiler

5% faster Chromium builds · **7% faster** on stdexec's slowest test

by reworking how Clang represents your code (2025). One engineer; everyone in this room waits less on every build.

Earlier I asked who replaces them. **This is one answer: the same apprenticeship, now inside the compiler.**

There are four C++ front ends. The most exacting one is opening up.

A front end is the part of a compiler that actually understands C++. There are only about four in the world, and you rely on all four.

EDG

The reference standard.
Quietly inside NVIDIA, Visual
Studio IntelliSense, Green
Hills, Coverity/Black Duck and
more, for nearly forty years.

Opening up · 2026

Clang / LLVM

Open source. The modern
workhorse behind much of today's
tooling.

Open source

GCC

Open source. Shipped Linux and
the free-software world for
decades.

Open source

MSVC

Microsoft's own front end.
Proprietary and closed.

Proprietary

A front end this exacting is a life's work. **Now it opens to everyone, and we are helping it endure.**



C++ Alliance Fellowship @ Compiler Research

Contributors: Vipul Cariappa, Ayokunle Amodu, Aditya Medhane, Muhammad Bassiouni, Peiqi Li, Ezzeldin Ibrahim, Sahil Patidar
Mentors: Vasil Vassilev, Aaron Jomy
Programme Administration: Martin Vassilev



About the Fellowship

Build the parts of Clang that matter next.

A remote, mentor-driven fellowship run by **Compiler Research** and sponsored by the **C++ Alliance**. It takes contributors from a first patch to production-quality code in the main branch of **llvm-project** — work that ships to everyone who uses Clang.

- **Upstream first.** Projects land as reviewed commits in **llvm-project** itself, no forks, no side branches.
- **Mentor-driven.** Weekly-week guidance from experienced LLVM maintainers and developers experience, tooling — **clang-tidy**, the static analyzer, clang-repl.
- **Global and remote.** Open worldwide, flexible participation, flexible contribution welcome.

Focus areas

C++20/23/26 language support; compile-time and memory performance; diagnostics and developer experience; tooling — **clang-tidy**, the static analyzer, clang-repl.

The 2026 cohort

7 fellow	5 mentors, 4 co-mentors	7 project tracks	6 mo full-time equivalent
----------	-------------------------	------------------	---------------------------

A year in the programme

Applications	Mar 15 – May 1
Community bonding	May 4 – 24
Coding I	May 25 – Jul 26
Midterm evaluation	Jul 27 – Aug 2
Coding II & final	Aug 3 – Oct 25

Location fair stipends, following the CScC purchasing power tables, paid in two halves at the midterms and final evaluations.

Ayokunle Amodu

Project

Optimize Usage of Source Locations in Clang Modules
Mentors: Vasil Vassilev, Aaron Jomy



Summary

Clang gives every token a source location in a fixed 32-bit address space, and large modular builds can exhaust it and crash. Much of the waste is duplication: when many modules include the same header, each loads its own copy of that file's source locations. This work deduplicates them in the module loader, so a file an earlier module already loaded is reused instead of recompiled.

Progress So Far

- Deduplication works end to end and is up for review on llvm-project, with the core mechanism in place and moving toward a merge.
- Compile time is negligible, peak memory rises a little from the small per-module mapping tables the rewrapping adds, which is next to bring down.
- Tested downstream at Apple, CERN, Google and Alibaba against their own module-heavy codebases, currently working through crashes from a large real-world build.

Background

Summary: Compiler engineer and LLVM Project member working across Clang, MLIR and ClangIR. "I love learning how compilers interact with hardware."
Education: Bachelor of Computing Science, University of Alberta, Canada, 2022 – 2026
Country: 🇨🇦 Canada
Email: ayokunle@gmail.com



Aditya Medhane

Project

Enhance Clang Diagnostics
Mentors: Vasil Vassilev, Aaron Jomy



Summary

Close gaps in Clang's warnings: fix diagnostics that miss real bugs, flag undefined behavior the C standard calls out, and turn useful opt-in warnings on by default. 26 upstream PRs in llvm-project so far.

Progress So Far

- Fixed `-Wunused-logical-pointer-compare` to fire through function references (merged).
- Added a diagnostic for conflicting linkage, UB per WG14 N3140 (targeting Clang 24).
- Enabled `-Wunused-template-ender` -Wall after 30 memory-overflow cleanup PRs (live on main); already driving fixes in Bitcoin Core and HarfBuzz.
- Next: moving typelike checks from **clang-tidy** into **Sema**.

Background

Summary: Open-source contributor across compilers and iOS; WWDC 2025 Swift Student Challenge winner.
Education: Bachelor of Technology in Computer Science, ABV-ITM Greater, India, 2023 – 2027
Country: 🇮🇳 India
Email: adityamedhane.dev@gmail.com



Muhammad Bassiouni

Project

Implement Missing C++26 Features in Clang
Mentors: Vasil Vassilev, Aaron Jomy



Summary

Bring language features to the Clang compiler, starting with `declarations` (C2y paper N3356) and continuing with clean template argument deduction (CTAD) for alias templates (P181480), each with diagnostics and a conformance test suite.

Progress So Far

- If `declarations` (N3356) implemented and merged into Clang 23.0.0git, routing and refactoring the existing C++ condition parser for C mode.
- Backported if `declarations` to every C mode down to C89 as a compatible extension, with new errors, warnings, and compatibility diagnostics for the C2y grammar pitfalls.
- Added `__extension__`, C23 attribute declarations, and static `assert` declaration in conditions — the first of these was not previously available in C++.
- Motivated extending Clang's `DiagCompat` compatibility-diagnostics infrastructure to C language modes, now available to all C diagnostics.
- 20+ line conformance test suite, plus `release notes`, `Language Extensions` docs, and the public C conformance page updated; feature-complete and ahead of current implementation is GCC.
- Currently working on CTAD for alias templates (P181480), toward full C++20 conformance.

Background

Summary: Software Engineer and Open Source Maintainer of the LLVM C Library, with nearly three years of experience in compilers, toolchain technologies, and backend development. Interested in working on cutting-edge, scalable services technology with global impact.
Education: Bachelor of Artificial Intelligence in Intelligent Systems, Menoufia University, Egypt, 2021 – 2025
Country: 🇪🇬 Egypt
Email: muhammad.a.bassiouni@gmail.com



Acknowledgements

This programme is funded by the C++ Alliance and run by the Compiler Research group. Everything on this poster reached users through the LLVM Project's open public review process: the work is the fellows', the bar it had to clear is the community's.

Thanks to

- Vasil Vassilev and Aaron Jomy, for mentoring every track on this poster week by week, from a first patch to a merged commit.
- Martin Vassilev, for running the programme — selection, evaluations and stipends — so the fellows could stay on the code.
- The LLVM reviewers and code-owners who read these patches across time zones, and the downstream teams who tested them against their own builds.
- CppCon, for the room and the conversation in front of this poster.

Partners



The C++ Alliance funds this fellowship outright: no fee, no equity, no strings on when the code lands.
This poster was created with 99igN by Mohamed Bassiouni

Peiqi Li

Project

Process High-Impact `clang-tidy` and Clang Static Analyzer Reports
Mentors: Vasil Vassilev, Aaron Jomy



Summary

Resolve high-impact issues in **clang-tidy** and the Clang Static Analyzer, focusing on false positives, false negatives, unmade fix-it hints and requests for new checks. This work also bridges some core **clang-tidy** capabilities directly into **clangd** for language server users.

Progress So Far

- **clangd Architectural Integrations:** Decoupled `mergeFixits()` into `clang::edit` to prevent syntax errors (#208632), enabled query-based custom checks (#216336), and added active `clang-tidy` line filtering (#202575).
- **High-Impact Bug Fixes:** Fixed `use-after-move` AST unwrapping (#199905), redundant-expression typos for false positives (#198485), and default `IgnoreExternC` loader protections (#199189).

Background

Summary: Open-source contributor interested in compilers. Currently a C++ Alliance research fellow working on LLVM tooling.
Education: Bachelor of Mathematics and Artificial Intelligence, Shanghai Jiao Tong University, China, 2024 – 2028
Country: 🇨🇳 China
Email: voyager.1pq@gmail.com



Ezzeldin Ibrahim

Project

Improve Clang Performance
Mentors: Vasil Vassilev, Aaron Jomy



Summary

Clang's compile time and peak memory drift upwards as the language grows: new features, added diagnostics and evolving usage patterns leave small inefficiencies behind that no single patch is blamed for, but that accumulate reduce over release into real cost for large C++ projects. This project hunts those hidden regressions systematically by benchmarking real-world C/C++ codebases across Clang 18 to trunk, bisecting each slowdown to its first-bad commit, and upstreaming fixes.

Progress So Far

- Optimized the fixable ones: 6 patches merged so far (typo-correction gating, ~1.2% on Boost.MPL; cached analysis-based warning policy), ~1.2% total frontend win.

Background

Summary: Open-source contributor working on compilers and developer tooling, LLVM contributor, contributing to the LLVM C Library and Clang. Currently a C++ Alliance research fellow.
Education: Bachelor of Computer Science, Zagazig University, Egypt, 2022 – 2026
Country: 🇪🇬 Egypt
Email: ezzelbrahim@gmail.com



Sahil Patidar

Project

Consistent Error Recovery Infrastructure
Mentors: Vasil Vassilev, Aaron Jomy



Summary

Interactive C++ compilation keeps compiler state alive across multiple inputs, unlike traditional batch compilation. When an input fails, changes made to the shared compiler state can remain behind and affect subsequent inputs, causing incorrect diagnostics, redefinition errors, or unstable behavior.

This project aims to improve Clang's error recovery by ensuring that failed inputs can be completely reverted to the last valid compiler state, while preserving all previously successful inputs and avoiding the need to restart or rebuild the compiler.

Progress So Far

- Investigated the memory-rewind prototype through testing, studying how failed PTUs leave invalid mutations in Clang's shared AST and semantic state, and how memory rewind affects compiler state. Collected cases of dangling reference crashes and incorrect diagnostics caused by failed inputs.
- Designed PTU-aware state tracking using `ASTMutationListener`, based on the observed failure cases and state mutations across incremental inputs.
- Implementing the concrete tracking and rollback system based on the gathered findings and design.

Background

Summary: Open-source contributor to LLVM, working across projects involving ORC-JIT, clang-repl, and other parts of the LLVM project.
Education: Bachelor of Technology in Computer Science and Engineering, VIT-Vellore Institute of Technology and Science India, India, 2019 – 2023
Country: 🇮🇳 India
Email: sahilpatidar60@gmail.com



Join the 2027 Cohort

Applications open March 15 – May 1, 2027

Programme & projects



https://compiler-research.org/2027

Ask us anything on Discord



https://discord.gg/9P8m3d4

How to apply

1. Pick a project from the open list.
2. Discuss your plan on the Compiler Research Discord.
3. Email your proposal to vvasil@llvm.org.

What a fellow gets

- Weekly-week mentorship from LLVM maintainers.
- A location-fair stipend, paid in two halves.
- Commits, RFCs and docs under your own name.

Sponsor or partner

Find a fellow, or bring a project to the programme: vvasil@llvm.org.

Part of the Compiler Research Institute — funded by the C++ Alliance — compiler-research.org

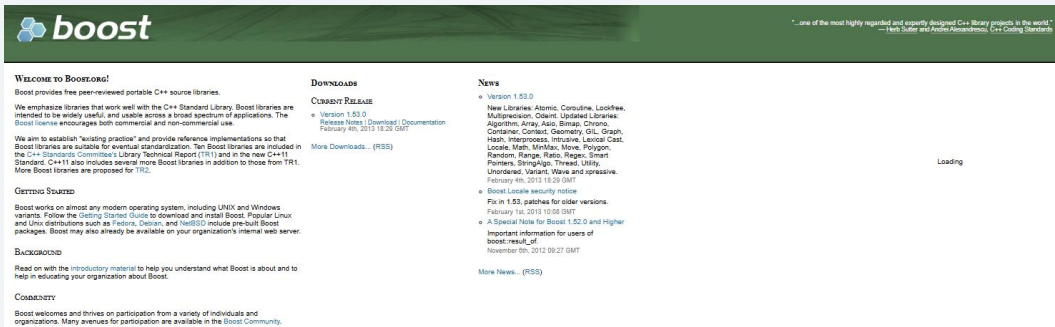


compiler-research.org

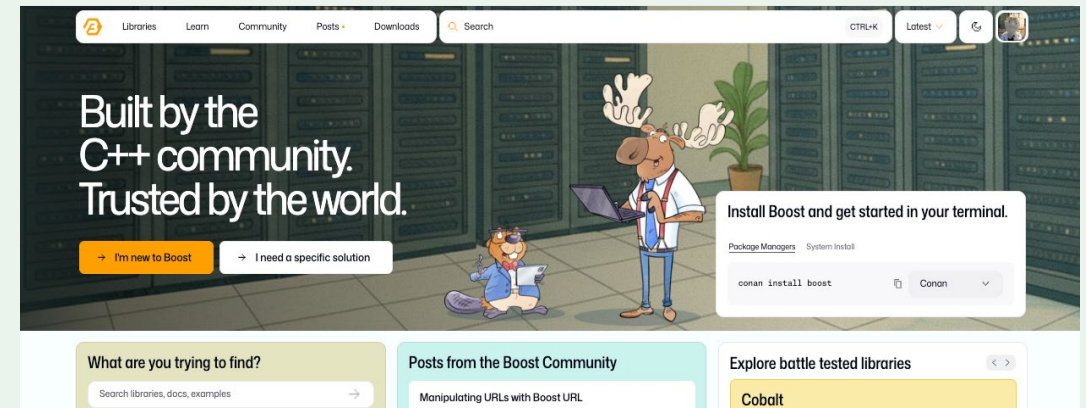
SPONSORED BY C++ ALLIANCE

We rebuilt the front door

BEFORE · boost.org, 2013



AFTER · rebuilt site + unified docs



A newcomer decides in five seconds whether to stay or leave for Rust. We fixed the five seconds.

And we are telling the story

The Boost Documentary

the people, the history, the
arguments, the craft on film

Internships & mentoring

apprenticeships, the guild way

The rebuilt front door

so the next newcomer stays

Fast libraries earn respect. Stories earn the next apprentice. We intend to do both.



And playing games



Boost.Documentary

#include <coder_community>

Screening

Wednesday, Sept 16

6:00 PM

Location

Colorado A

Audience Preview

Meet the Filmmakers

Alen Calich

Imani Dean

Ray Nowosielski

Elly Michaels

Collier Landry

Q&A Session

Join us for a 30-minute Q&A immediately following the film with the director and special guests in person.

Boost.Documentary

#include <coder_community>



Screening

Wednesday, Sept 16 · 6:00 PM

Location

Colorado A (Audience Preview)

Meet the film Makers

Alen Calich, Imani Dean,
Ray Nowosielski, Elly
Michaels, Collier Landry

Q&A

Join us for a 30 minute Q&A
immediately following the film with the
director and special guests in person.



One name



Arnaud Becheler

Scientific computing · Boost.Graph · C++ Alliance

Nearly ten years a Boost user in science, on the outside of the glass.

Then he crossed over: brought his work, took the reviews, kept coming back.

Mentored by Joaquín. Today he helps maintain Boost itself.

Boost is where you become the engineer you want to be.

While you're here, catch these

Sessions of related interest to Boost, C++ compilers, and the C++ Alliance.

Tue • Sep 15
4:45 PM MDT

Matheus Izvekov From User to Contributor: fixing bugs and adding features in Clang

Wed • Sep 16
9:00 AM MDT

Mateusz Pusz Awaiters and Awaitables: build a coroutine awaiter from scratch

Thu • Sep 17
12:30 PM MDT

Arnaud Becheler The Barrier Is a Myth: becoming a Boost maintainer

Thu • Sep 17
2:00 PM MDT

Alan de Freitas Reliable C++ reference docs from the Clang AST (MrDocs)

Fri • Sep 18
10:30 AM MDT

Matt Borland IEEE 754 Decimals for C++: the Boost.Decimal library

If a thread in this talk grabbed you, **there is a whole session on it this week.**

Come build the next one



Use the libraries



File issues



Review code



Write a library



Be a review manager



Alliance internships

Boost is where you become the engineer you want to be.

Any Questions?



harry@cppalliance.org
Harry Bott, CEO, The C++ Alliance